KOIOS · THE PLAYGROUND · COMPANION 0.19.0

Mounting the folder on the compute

The code stays on your computer; the GPU reads it there, through the Companion's Cloudflare tunnel, as if it were its own disk. Saves are there in under a second, results come back as they are written, and checkpoints go where the data is kept. What was built, why, how it was checked and measured, and how to try it.

Branch claude/epic-darwin-18jc7s at 59949ff · 2026-10-20 · printed from docs/compute-mount.md and docs/compute-mount-process.md

Part 1 · What it is, and how to use it

- 1. Turning it on
- 2. What goes where
- 3. How it reaches your computer
- 4. Two modes
- 5. When your computer sleeps, or is far away
- 6. A tunnel of your own, with Argo Smart Routing
- 7. Measuring it over a real tunnel
- 8. Reference

Part 2 · How it was built, checked and measured

- 1. The ask
- 2. Before: two ways of copying
- 3. The choices
- 4. How it fits together
- 5. The order of work, and what each run found
- 6. What was measured
- 7. The tests
- 8. Which limits are the mount's, and which the tunnel's
- 9. Trying it yourself, step by step

PART 1

What it is, and how to use it

Your project's files can stay on your computer while its code runs on Colab's GPU or a GPU server of yours. Before Companion 0.19.0 the code got there as a copy: before each run the changed files were copied over, and after it the results were copied back. With the **mount**, the GPU machine reads the folder where it is, the way a notebook reads Google Drive once it is mounted. Nothing is copied before a run and nothing is brought back after it.

How it was built, checked and measured, with the step-by-step test plan, is in compute-mount-process.md. Both documents are printed as one PDF, `docs/compute-mount-process.pdf` (`node scripts/compute-mount-pdf.mjs`).

This needs Koios Companion **0.19.0 or later** on the computer that has the files, and a compute link (compute-link.md).

1. Turning it on

Storage & compute → **Sync** → **Mount the folder on <machine>**. The setting is kept with the project. Whenever the machine doesn't have the folder mounted (a new Colab runtime, say), the page starts the mount again. The chip above the terminal reads **Mounted on Colab ⇄ live** while it is up. From a terminal on the computer with the files:

```
koios mount            # fuse where the machine allows it, else a live mirror
koios mount --mode mirror
koios unmount          # back to copying before each run
koios status           # says whether it is mounted, and connected
```

While the folder is mounted, `koios run` and the page's ▶ **Run** skip the copy, and **Copy now** says there is nothing to copy.

2. What goes where

A FILE…	LIVES	REACHES THE OTHER SIDE
of your project (code, configs, notebooks, small results)	on your computer	the machine reads it there; a save on either side is on the other within about a second
the project's sync rules leave out (`data/`, `wandb/`, `*.pt`, `*.ckpt`, `*.safetensors`, by default)	on the machine's own disk, under the mount	put in the project's **data place** (Google Drive by default, or a computer of yours) a few seconds after it is closed
over 100 MB, written on the machine	on the machine's own disk	the data place, never the tunnel
a cache (`__pycache__`, `.ipynb_checkpoints`, `.reader/`)	on the machine	nowhere

So a checkpoint written to `checkpoints/model.pt` is written at local disk speed on the GPU machine and goes to wherever you keep data. It never crosses the tunnel to your laptop. The rules are the ones under

Not copied in the Sync pane, and they mean the same thing they did before.

3. How it reaches your computer

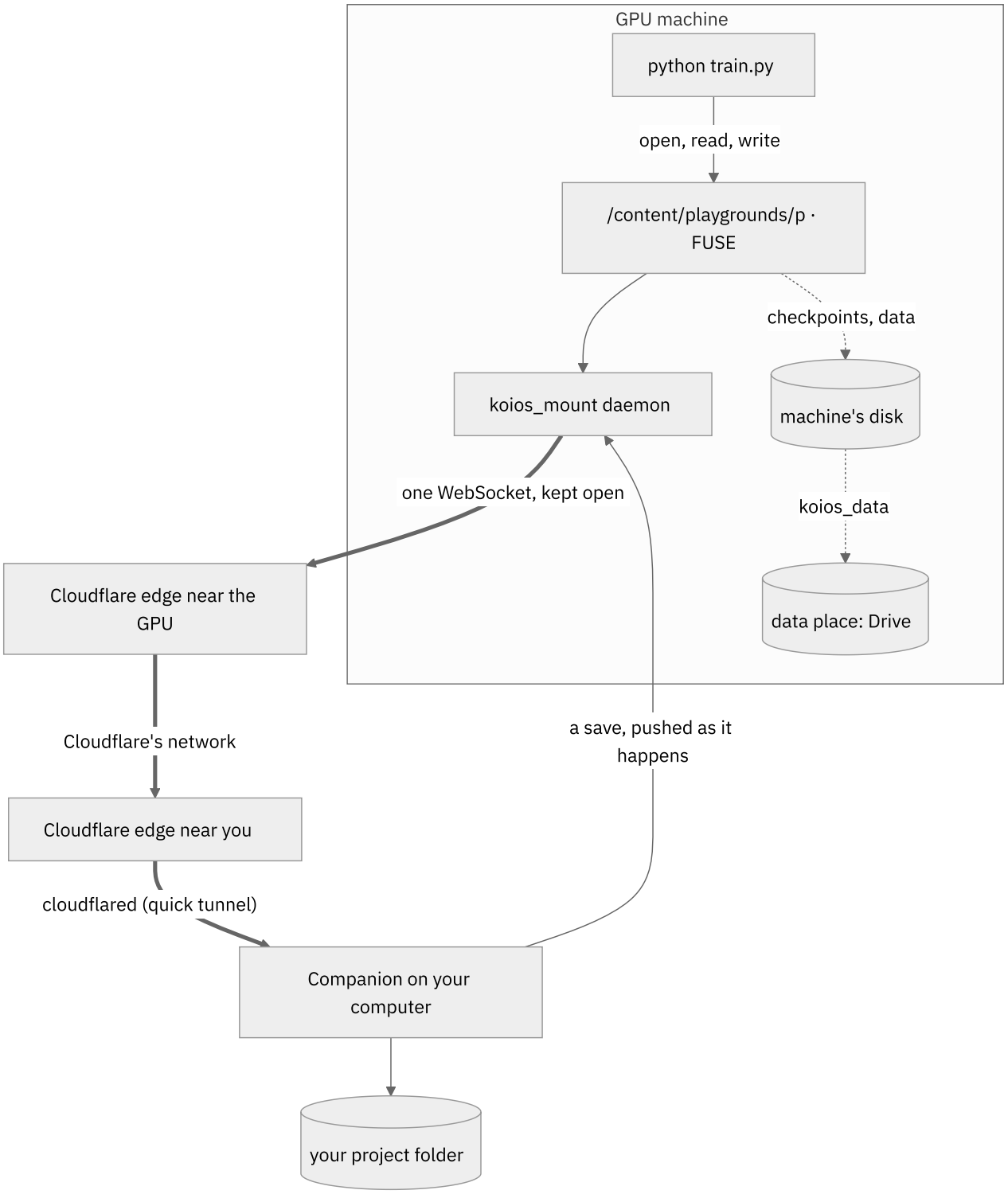


Figure 1. The machine dials out to the nearest Cloudflare edge. Cloudflare carries the connection to the edge nearest your computer, and the Companion's `cloudflared` *hands it on. The browser and the Worker are not on the path, and neither machine accepts a connection from the internet.*

- **One socket, kept open.** The daemon on the machine (`koios_mount.py`) holds one WebSocket to `/companion/mount/socket` on the Companion's tunnel. It doesn't open a connection per file or per run. If the socket drops, the daemon reconnects with backoff, and the Companion sends the tree again on reconnect.
- **The small files first.** On connecting, the Companion sends the folder's tree. Every file of 2 MB or less then comes over in a few gzip'd archives (Python files first), so imports and configs are local from the start. Bigger files are fetched in 4 MB pieces the first time they are opened, then kept in the cache.
- **Your saves are pushed, not polled for.** While a machine is connected, the Companion looks over the folder twice a second (stat calls only) and pushes each change straight away. A change of 512 KB or less carries the file itself, so the machine never has to ask for it. A write from the machine is not echoed back to it.
- **Writes go back on close.** A file written on the machine is sent when it is closed (or `fsync`'d), in pieces. The Companion writes it beside the original and swaps it in, so a reader on your computer never sees half a file.
- **The key opens one folder.** The machine holds a key made for this project's folder, kept in `~/.reader-companion/mounts/` (0600). It never gets the Companion's token. The socket refuses paths outside the folder, links that lead out of it, and paths the rules say are the machine's own. **Unmount** forgets the key, which closes every socket that used it.

4. Two modes

- **fuse** (Colab and most servers: `/dev/fuse` and libfuse 2). This is a real mount, and it is lazy: only what is opened is fetched. The first mount on Colab runs `pip install fusepy` and, as root, `apt-get install fuse libfuse2`. Allow about a minute for that.
- **mirror** (containers that don't allow FUSE). The folder is a real directory that the daemon keeps current both ways over the same socket. Every file is on the machine, but your saves still arrive as you make them, and edits made on the machine go back within a second.

In fuse mode, what was in the folder before the mount is sorted when the mount starts:

- **Copies of your files** from earlier pushes are dropped, because the mount is current.
- **The machine's own files** (checkpoints, the helpers under `.reader/`) are kept under the mount.
- **Anything else a run left there** is sent back to your computer.

Unmounting puts the machine's own files back in the folder, so copy-before-run works again as it did. A mirror leaves what was there as it was, and sends only what changes after it starts.

5. When your computer sleeps, or is far away

The mount is built so a laptop that sleeps, loses its network, or sits on another continent slows a run down at worst. It never stops one.

- **Everything comes over in the background.** After the first archives, the daemon fetches the rest of your files one at a time, smallest first. That covers each file up to 200 MB, and up to 4 GB in all (`KOIOS_MOUNT_FILL_FILE_MB`, `KOIOS_MOUNT_FILL_MB`; `0` turns it off). After that the machine reads nothing from your computer, so a sleep mid-run costs nothing for reads.

- **A read that does need your computer waits instead of failing.** If a file isn't on the machine yet and your computer can't be reached, the read waits up to 15 minutes for the connection to come back (`KOIOS_MOUNT_WAIT_S`). The program is held up, not crashed. Listings and file sizes never wait: they come from the tree the machine already has.
- **Writes queue.** A file written while your computer is away is kept on the machine, and the program's `close()` succeeds. The queue is saved on disk, so it survives a restart of the daemon. It is sent as soon as the socket is back, and the daemon redials with backoff. If your computer saved the same file in the meantime, the machine's version wins, and the daemon logs it.
- **Your computer is kept awake while it is mounted.** While a machine is connected, the Companion holds the system's own lock against sleep: `caffeinate -i -s` on macOS, `systemd-inhibit` on Linux, `SetThreadExecutionState` on Windows. It lets go when the last machine disconnects. The display can still sleep. A laptop on battery with its lid closed sleeps anyway; nothing a program does changes that, and the three points above cover it. To turn this off, put `"keep_awake": false` in `~/.reader-companion/config.json`.
- **Notebook cells run in the project's folder.** While the folder is mounted, the page moves the notebook's kernel into it, quietly, once per kernel: no output, and no cell number used. So `import mymodule` and relative paths work from a cell as they do from the console.
- **Distance.** The first read of a big file costs a round trip through Cloudflare: tens of milliseconds within a region, a few hundred across an ocean. Argo Smart Routing (below) shortens that middle leg. Datasets still belong in the data place (`koios_data.path(...)`), not in the project folder.

6. A tunnel of your own, with Argo Smart Routing

The Companion's default tunnel is a free **quick tunnel**. It needs no account, but it gets a new `….trycloudflare.com` address at every start and can't use Argo. **Argo Smart Routing** is Cloudflare's paid routing. It carries traffic between Cloudflare's data centres on the paths it currently measures as fastest. For a mount, that is the leg between the edge near the GPU machine and the edge near your computer. To use it, give the Companion a **named tunnel** on your own Cloudflare account:

1. You need a domain on Cloudflare (any plan) with billing set up; Argo is billed per month and per GB.
2. Create an API token at dash.cloudflare.com → My Profile → API Tokens. It needs these permissions, and can be limited to the one zone: - Account · Cloudflare Tunnel: Edit - Zone · DNS: Edit - Zone · Zone Settings: Edit - Zone · Zone: Read
3. On the computer with the files:

```
reader-companion tunnel setup --hostname koios.example.com --api-token <token>
reader-companion start            # or reopen the Koios app
reader-companion tunnel status    # named tunnel, Argo on
```

Setup does the following through Cloudflare's API:

- creates (or reuses) a tunnel named `koios-<computer id>`;
- points `https://koios.example.com` at the Companion;
- adds the DNS record;
- turns Argo Smart Routing on for the zone;
- keeps the tunnel's own token in the Companion's settings (0600).

The API token is not saved. The Companion runs `cloudflared tunnel run` with the tunnel's token in its environment, not on the command line. Mounts then dial the fixed address, so a restart of the Companion no longer changes it.

If Argo can't be turned on (no billing, or a token without Zone Settings: Edit), setup says why and the tunnel works without it. You can turn Argo on later under your domain → Traffic → Argo Smart Routing. `reader-companion tunnel quick` goes back to quick tunnels.

7. Measuring it over a real tunnel

`companion/tests/live_tunnel.py` serves a folder and mounts it through a real tunnel, measures it, and prints:

- how long the mount takes to be ready;
- a cold and a cached read of a 6 MB file;
- how long a save takes to reach the other side, each way;
- the round trip of a read.

```
cd companion && python3 -m tests.live_tunnel                    # a quick tunnel
KOIOS_LIVE_HOSTNAME=koios.example.com KOIOS_LIVE_TUNNEL_TOKEN=<tunnel token> python3 -m
tests.live_tunnel   # yours, with Argo
```

Both ends run on one computer, but the daemon dials the tunnel's public address, so every byte goes out to Cloudflare and back in. It needs `cloudflared` to reach Cloudflare's edge: TCP or UDP port 7844 out, and DNS for `argotunnel.com`. Many CI sandboxes block that; the one this feature was built in did. There, `cloudflared` was handed a `trycloudflare.com` address (that request is ordinary HTTPS) but couldn't open the tunnel itself.

8. Reference

FILE	WHAT
`companion/reader_companion/koios_mount.py`	The machine's side: the socket, the cache, the FUSE operations, the mirror, the data shipper, `start` / `stop` / `status`. Standard library, tornado and fusepy.
`companion/reader_companion/mount.py`	The Companion's side: the keys, the socket (`MountSocket`), the watch, and starting the daemon there through the link's kernel.
`companion/reader_companion/extension.py`	`ComputeHandler`: `POST companion/compute` with the action `mount`, `unmount` or `mount-status`. `GET` says whether a machine is connected.
`companion/reader_companion/awake.py`	This computer kept from sleeping while a machine has a folder mounted.
`companion/reader_companion/cloudflare.py`, `tunnel.py`	`reader-companion tunnel setup`: a named tunnel at your hostname, with Argo Smart Routing; the Companion runs it instead of a quick tunnel.

FILE	WHAT
`companion/reader_companion/compute.py`	`koios mount`, `koios unmount`; `koios run` and `koios push` skip the copy while the folder is mounted.
`src/lib/computeLink.ts`	`mountThroughCompanion`, `unmountThroughCompanion`, `linkState`.
`src/components/PlaygroundWorkspace.tsx`	The switch in the Sync pane, the **Mounted** chip, and runs that skip the copy.
`companion/tests/live_tunnel.py`	The mount over a real tunnel, measured (section 7).
`companion/tests/test_mount.py`, `test_awake.py`, `test_cloudflare.py`, `scripts/compute-link.test.mjs`	Tests. These include a real FUSE mount and a mirror, each edited on both sides, and the page's one call against a real Jupyter server standing in for the GPU box.

PART 2

How it was built, checked and measured

The companion to compute-mount.md, which describes *what* the mount is and how to use it. This one is the process: what the code did before, the choices made and the ones turned down, how the pieces fit, what each test run found, what was measured and what it showed, which limits belong to the mount and which to the tunnel, and the steps to try it yourself. It is written so that someone who wasn't in the session can retrace the work and check what it claims.

The branch is `claude/epic-darwin-18jc7s`. `docs/compute-mount-process.pdf` prints both documents as one PDF; run `node scripts/compute-mount-pdf.mjs` to print it again.

---

1. The ask

1. *When I code on my laptop and use Colab's GPU, is the code copied there first?* Yes. Section 2 shows how.
2. *Can the connections we already have run the code on that machine without making a whole copy of the codebase there?* Yes, by mounting the folder. That is the build.
3. *Can Cloudflare's network make it feel like one device?* The mount runs over the Companion's Cloudflare tunnel, keeps one socket open, and pushes changes as they are saved (section 4).
4. *Checkpoints should go where the data is kept.* They stay on the GPU machine's disk and go to the data place (section 4.3).
5. *Test it over a real tunnel, handle the limits gracefully, and set up Argo Smart Routing.* Sections 6 to 8.

2. Before: two ways of copying

When a project's files were in one place and its code ran in another (split mode, `filesAreOnMachine` false), every command in the console copied the folder first and brought results back afterwards. There were two paths:

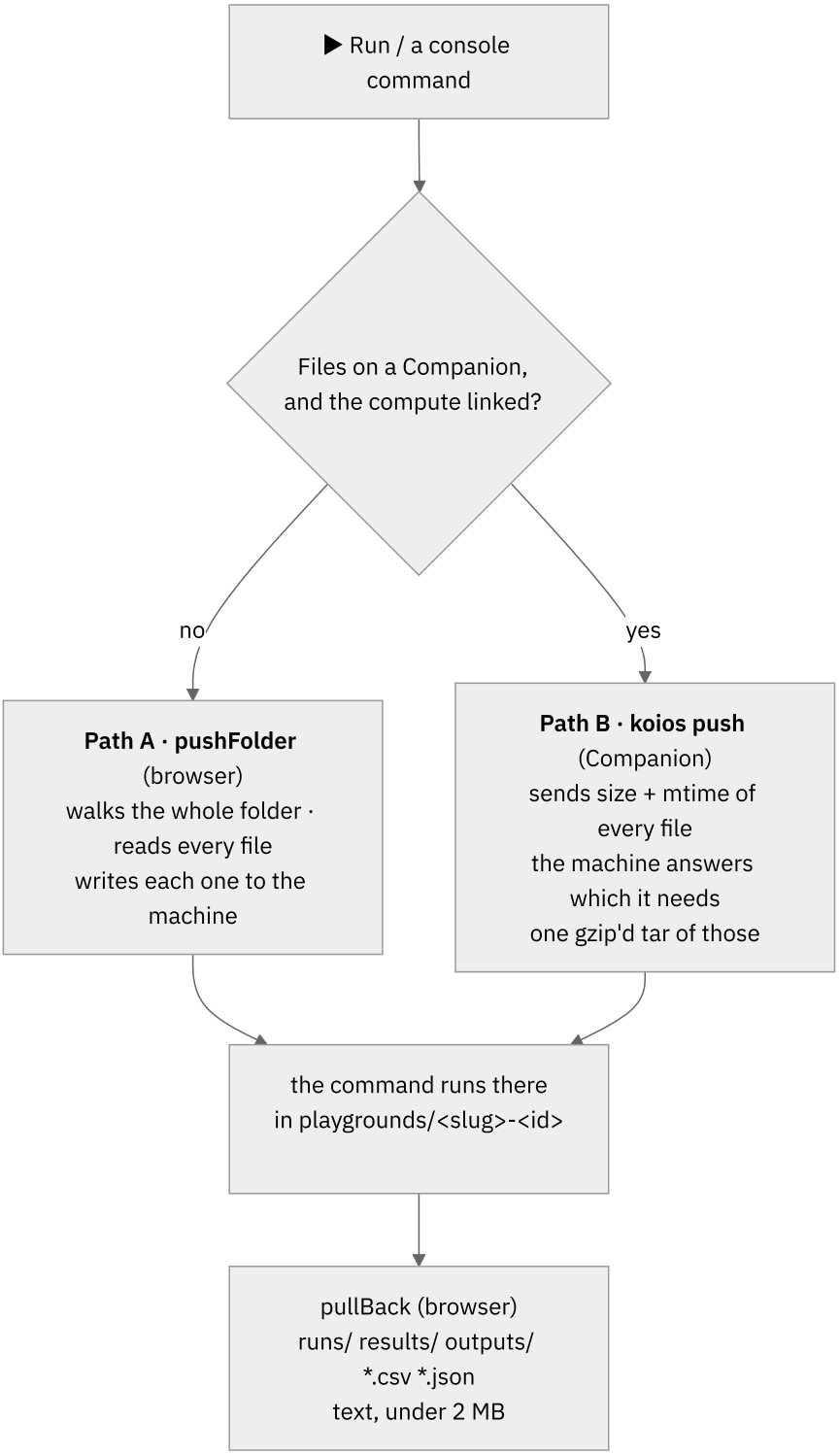


Figure 1. The copy before the mount. Path A sends every file every time (text only, 2 MB at most each, at most 400 files and six folders deep). Path B sends only what changed, but it still opens a connection to the machine for each command. Either way, notebook cells got no copy at all; only console commands did.

	PATH A (BROWSER)	PATH B (COMPANION)
Used when	Files in Drive, in the browser, on a plain Jupyter server, or on an unlinked Companion	Files on a Companion with a compute link (0.17.0 on)

	PATH A (BROWSER)	PATH B (COMPANION)
What goes over	Every file, every time	Files whose size or modification time changed
How	One request per file; on Colab, one kernel `execute_request` per file, the content in base64	A tar over Jupyter comm messages on one kernel socket
Limits	Text only, 2 MB a file, 400 files, 6 folders deep	100 MB a file, unless `--big`
Results back	`pullBack`, text only, under 2 MB	the same, through the browser

3. The choices

QUESTION	CHOSEN	TURNED DOWN, AND WHY
How does the machine see the folder?	**A FUSE mount**, with a live two-way mirror where FUSE isn't allowed	*A Python import hook*: only `import` in the notebook kernel would see the files. `python train.py`, `open()`, shells and agents would not. *sshfs*: needs SSH into the laptop from the internet.
Which side dials?	**The machine dials the Companion**	The laptop can't accept connections from the internet, and Colab's runtime only takes requests through its own proxy. The Companion already has an HTTPS address: its tunnel.
Over what?	**One WebSocket, through the Companion's Cloudflare tunnel**	*Through the browser* (the page's kernel socket): the run would depend on a tab staying open. *Through the Worker*: an extra hop, and the Worker has no disk. *Jupyter comms from the Companion* (as `koios push` does): the Companion would have to keep a kernel socket per machine, renewing Colab's hourly pass.
Who notices a change?	**The Companion pushes it** (it looks over the folder twice a second, stat calls only)	*The machine polling*: a round trip through Cloudflare for every look.
What credential does the machine hold?	**A key for one folder** (`mounts/<hash>.json`, 0600)	*The Companion's token*: it opens every folder, terminals, and the update route.
Where do checkpoints go?	**The machine's own disk, then the data place** (koios_data)	*Back to the laptop*: gigabytes through the tunnel to a disk that wasn't chosen for data.
What decides what is "data"?	**The project's existing sync rules** (`data/`, `*.pt`, ...)	A second list to keep in step with the first.

4. How it fits together

4.1 The pieces

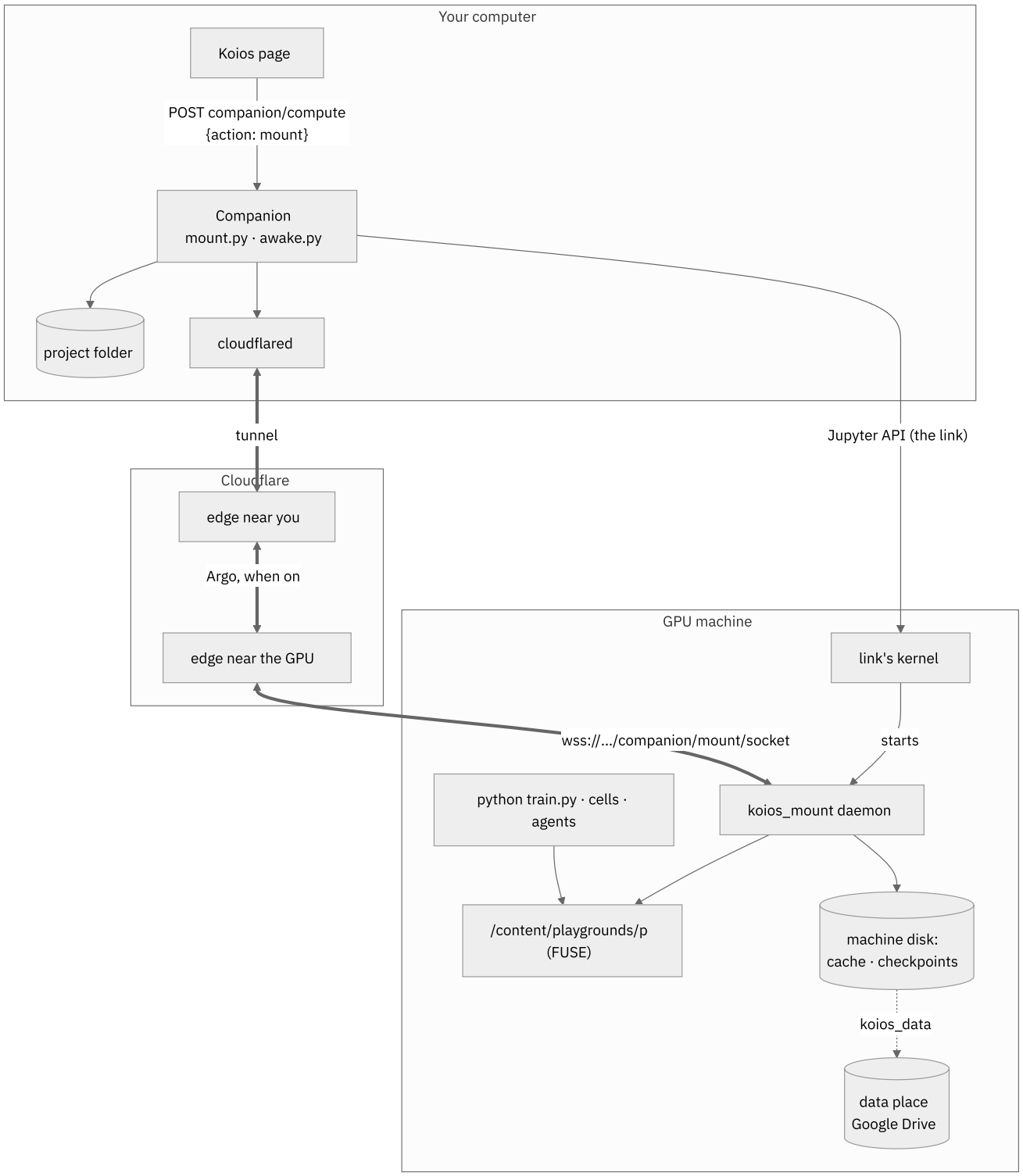


Figure 2. The page makes one call. The Companion starts the daemon through the compute link's own kernel; from then on the daemon and the Companion talk directly, through Cloudflare. The page and the browser are not on that path.

4.2 Starting a mount

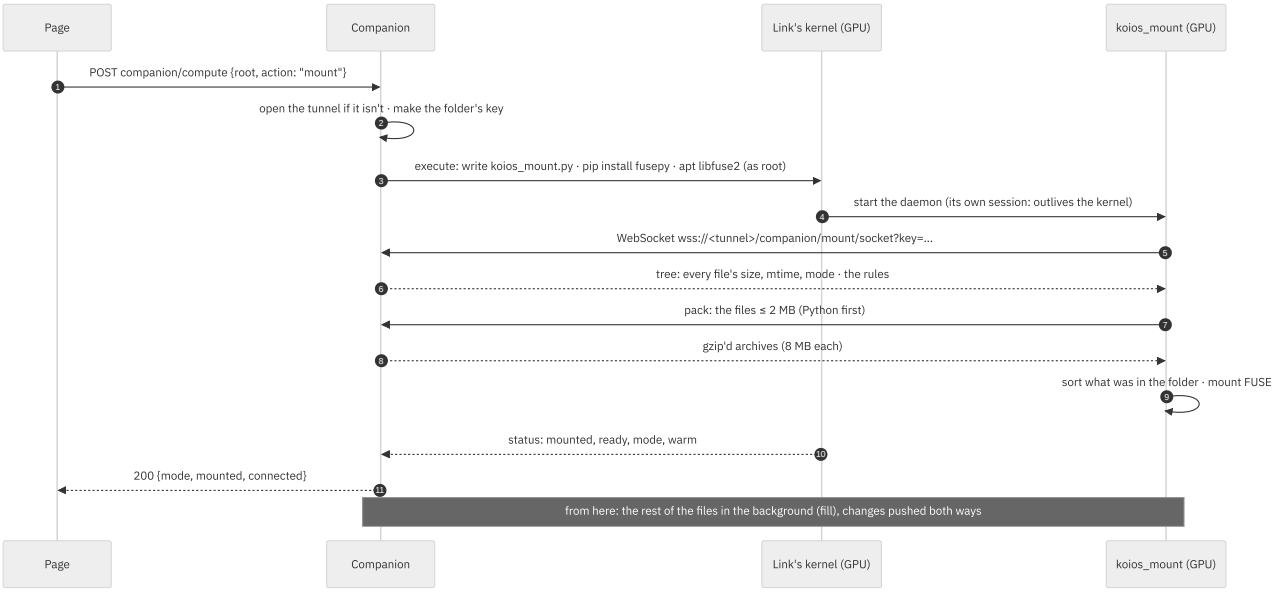


Figure 3. One call from the page, about a second on a fast link (section 6). The first time on Colab, add the minute that installing fusepy takes.

4.3 Where a write goes

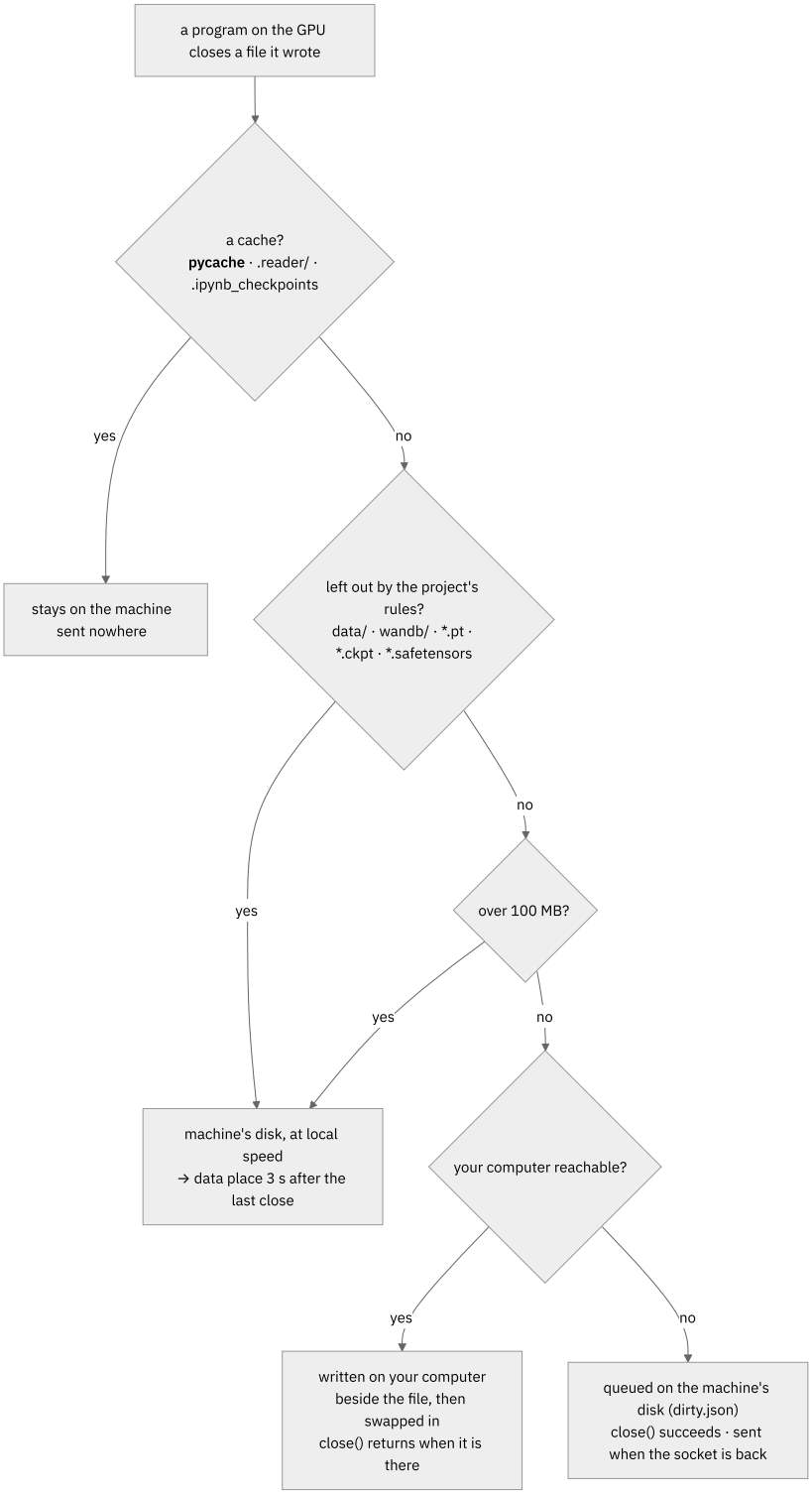


*Figure 4. Every write ends in one of four places, and none of them is "lost". The rules are the ones under **Not copied** in the Sync pane, so they mean what they meant before.*

4.4 A save on your computer

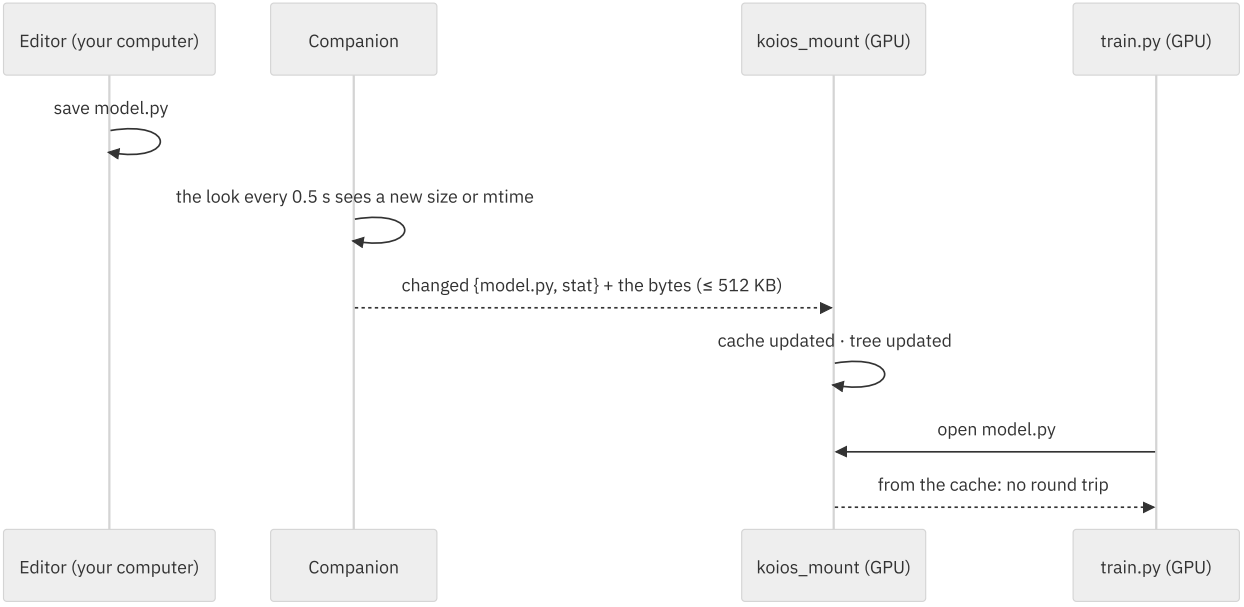


Figure 5. A change is pushed with the file itself, so by the time anything on the machine opens it, nothing needs asking. Section 6 shows this takes 0.34 to 0.49 s; most of that is waiting for the half-second look.

4.5 When the connection goes

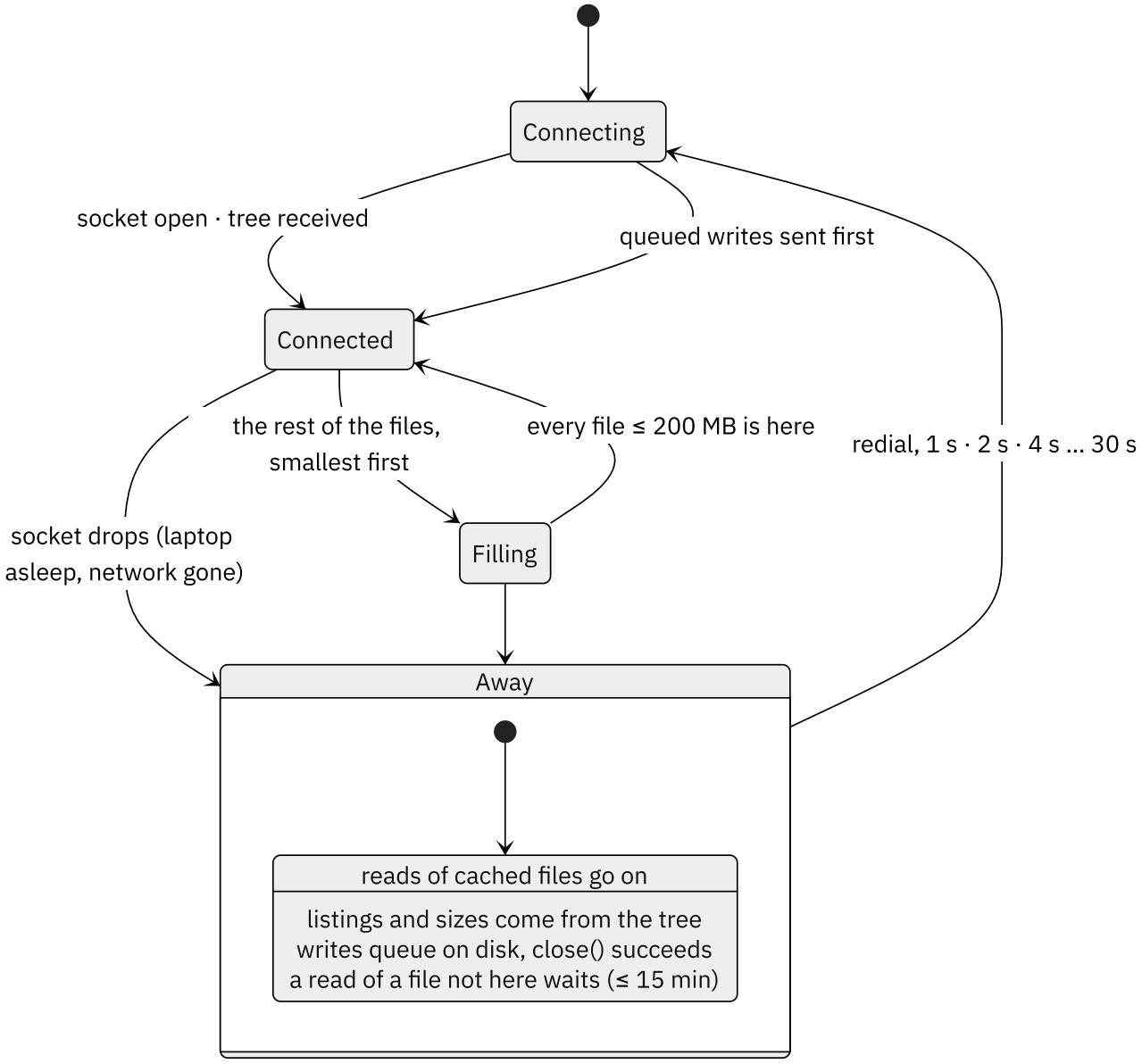


Figure 6. The daemon's states. "Away" covers both a sleeping laptop and a dropped tunnel, which look the same from the GPU.

5. The order of work, and what each run found

Each step was chosen so the next could be checked against something real.

#	STEP	WHAT IT FOUND
1	Traced the copy paths (an Explore agent over `playground.ts`, `PlaygroundWorkspace.tsx`, `compute.py`, `colab.ts`)	Section 2. Notebook cells never got a copy; only console commands did.
2	Checked whether FUSE works in the sandbox: `apt-get install fuse libfuse2`, `pip install fusepy`, a hello-world mount	`/dev/fuse` was there and a mount worked, so every test below is a **real** FUSE mount, not a stand-in.

#	STEP	WHAT IT FOUND
3	Wrote `koios_mount.py` (the GPU side) and `mount.py` (the Companion side), then `tests/test_mount.py`: both ends on one computer, the daemon in a process of its own	The first run failed: the handshake answered **500**. `MountSocket.write()` shadowed tornado's `RequestHandler.write`. Renamed to `write_piece`.
4	Data place test: a checkpoint written on the mount must reach a stand-in Drive folder	Two bugs. (a) A key starting with `-` was read by argparse as a flag; the daemon now gets `--key=…`. (b) The status never said where the data went, because nothing rewrote it after a send; the shipper now calls back.
5	The whole Companion suite	Passed on its own, failed in the suite. An earlier test left the process in a deleted directory, so `os.getcwd()` inside koios_data raised. The daemon now `chdir`s to its own state folder. The same run's log showed `Uncaught exception from FUSE operation lock`: fusepy takes **any attribute named after a FUSE operation** as that operation, and `Ops.lock` was a `threading.RLock`. Renamed to `guard`, so the kernel's own local locking applies.
6	The page's one call against a real Jupyter server standing in for the GPU box	It passed the first time once the test waited long enough (tornado's test client stops at 5 s).
7	Review of the socket's path checks	A symlink inside the folder pointing at `/etc/hostname` would have been served. Both `scan` and `path` now refuse links that lead out of the folder, and there is a test for it.
8	Review of `close()` while away	`flush` raised EIO when the laptop was unreachable, which would crash a training script writing a log. It now leaves the file queued and returns success.
9	"Away" test: the socket closed under a live mount, a write made, the server started again	FUSE passed. The **mirror failed**: a file written while away was taken for "something that was there before the mirror started" and never sent. That baseline now applies to the first tree only.
10	Background fill, reads that wait, a durable write queue, keep-awake, notebook cells in the folder, named tunnel and Argo	Tests for each (section 7). In the away test, a 3 MB file never opened was readable with the laptop gone.
11	A real tunnel from the sandbox (`tests/live_tunnel.py`)	**Could not be done here**. Cloudflare handed out the address (`agenda-virginia-pic-packed.trycloudflare.com`), because that is an HTTPS call. But `cloudflared` couldn't resolve `_v2-origintunneld._tcp.argotunnel.com` or open TCP 7844 to the edge. That is the sandbox's network, not the code.
12	Measured copy against mount, with simulated distance (`tests/bench_mount.py`)	Section 6.
13	The suite on Python 3.9 (the oldest the Companion supports, and the Mac's Homebrew Python)	The mount's tests passed. Seven tests in two older modules failed. `mover.py` packed with `tarfile.open` in streaming gzip mode with a `compresslevel`, which needs 3.12; it now gzips the stream itself. `test_paper.py` used a union type (`bytes` or `str`) in an annotation evaluated at run time; it now has `from __future__ import annotations`.

#	STEP	WHAT IT FOUND
14	The suite run again and again	`ThroughTheLink` failed about once in six runs: the daemon mounted, but the Companion was told it hadn't. A start/stop loop (60 runs) showed why. Several of the daemon's threads wrote `status.json` through **one shared part file**, so it could end up half one write and half the other, and unreadable. Each thread now writes a part file of its own.
15	The same loop, timing `stop`	**Every stop took 10 s** and ended in SIGKILL. libfuse takes SIGTERM for itself and acts on it only at the next request. And the daemon, once it did exit, stayed a zombie of the process that started it, so `stop` thought it was still running. Now the daemon ends by itself when its mount goes, a thread reaps it, and `stop` treats a zombie as gone. Stop takes 0.05 s, and the suite runs in about 60 s instead of 140 s.

6. What was measured

`companion/tests/bench_mount.py` runs both ways on one computer:

- For the copy, a Jupyter server stands in for the GPU machine, and the copy is `koios push` (what `koios run` does before every command).
- For the mount, the Companion's socket serves the folder and the daemon mounts it with FUSE.
- Between them sits a proxy that holds every byte back for a fixed time each way, for a round trip of **0, 50 or 150 ms**. Those stand in for a machine in the same city, on the same continent, or across an ocean. The proxy limits latency, not bandwidth, and it is not Cloudflare.
- The project is 300 small Python modules (12 KB each) that a package imports, and a 6 MB file.
- The raw numbers are in `docs/compute-mount-bench.json`; run `python3 -m tests.bench_mount` to measure again.

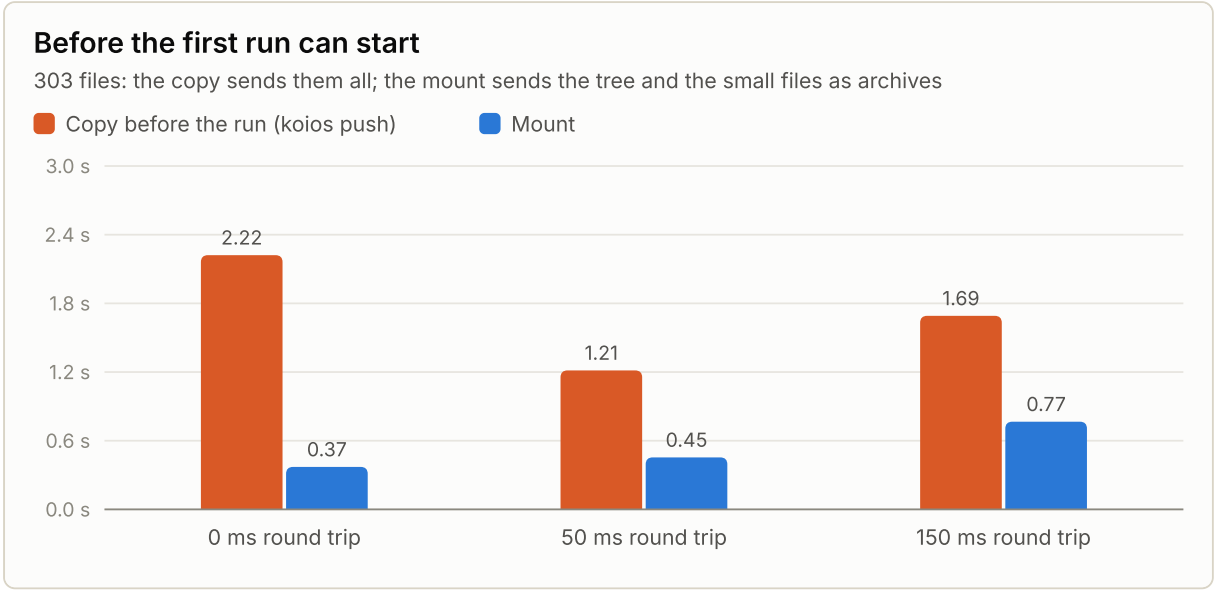


Figure 7. Before the first run can start. The copy sends all 303 files, and the first one also starts a kernel on the machine. The mount sends the tree and the 302 small files as archives. The copy's 2.2 s at 0 ms includes that machine's first kernel start, which was slower than the later ones.

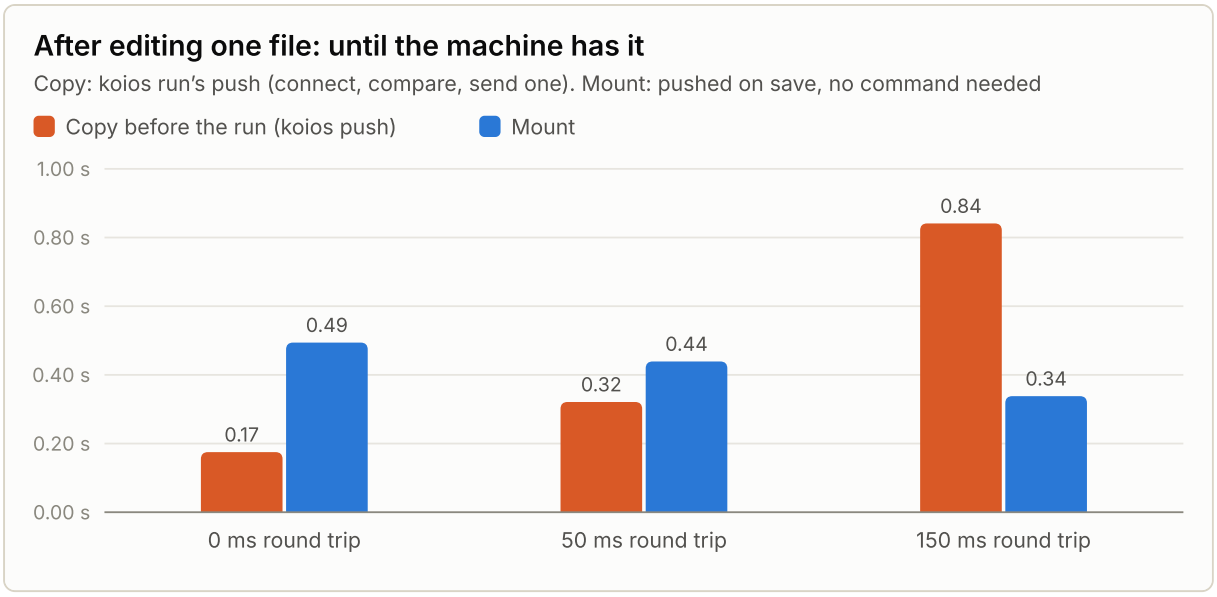


Figure 8. After editing one file. The copy is `koios run` 's push: connect, compare 303 files, send one. It grows with distance because each command opens a new connection, several round trips. The mount's time is the half-second look on your computer plus one trip, so it barely moves with distance. The bigger difference doesn't show in the chart: with the copy someone has to run it, and with the mount nobody does. At 0 ms the copy is quicker, because the look is on a 0.5 s timer.

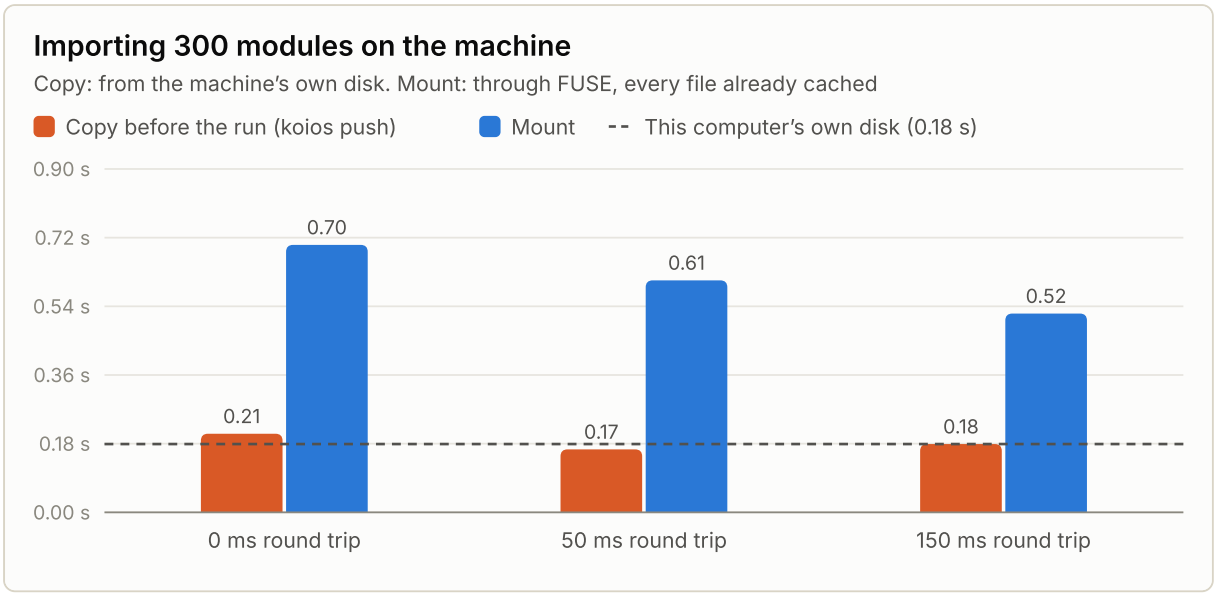


Figure 9. The cost the mount adds: importing 300 modules takes about 0.5 to 0.7 s through FUSE, against about 0.2 s from the machine's disk. The files are all cached by then, so the extra is FUSE itself: Python looks up each module in several places, and every lookup is a call into the daemon. The cost doesn't grow with distance (the 150 ms row is the fastest), and it is paid once per process start, not per step of a training loop.

ROUND TRIP	COLD READ OF 6 MB THROUGH THE MOUNT	SAME FILE, CACHED	A WRITE ON THE MACHINE, ON YOUR COMPUTER BY
0 ms	0.31 s	0.011 s	the time `close()` returns

ROUND TRIP	COLD READ OF 6 MB THROUGH THE MOUNT	SAME FILE, CACHED	A WRITE ON THE MACHINE, ON YOUR COMPUTER BY
50 ms	0.32 s	0.013 s	the time `close()` returns
150 ms	0.50 s	0.011 s	the time `close()` returns

Table. A cold read costs the round trips of two 4 MB pieces, plus the work of copying them. Once read, a file is read at disk speed. A file written on the machine is sent while it is being closed, so `close()` returning means it is on your computer. That is why the "write back" time measured as 0.

What the numbers don't cover: a real Cloudflare path (its own latency, and its throughput for large reads), Colab's disk, and a laptop on Wi-Fi. `tests/live_tunnel.py` measures the first one, and section 9 gives the steps.

7. The tests

TEST	WHAT IT PROVES	REAL OR STAND-IN
`test_mount.Mounted` (FUSE) and `.Mirrored`	Your files are there and current. `python main.py` runs from the mount. A save here is there within seconds, and a write, rename or delete there happens here. Checkpoints and `wandb/` stay on the machine and reach the data place. Caches go nowhere. What was in the folder before is sorted.	Real FUSE, real WebSocket, real processes
`…test_a_write_while_your_computer_is_away…`	With the socket gone, a never-opened 3 MB file is readable (fill), and a write succeeds and arrives when the socket is back	Real
`…test_stopping_gives_the_machines_own_files_back…`	Unmount puts the machine's files back, and a forgotten key is refused with 403	Real
`test_mount.SocketRefusals`	`../x`, absolute paths, `.git/`, `data/`, `*.pt`, and links out of the folder are refused	Real
`test_mount.ThroughTheLink`	The page's one call starts the mount on a Jupyter server through the link's kernel. GET reports it connected. Copy says there is nothing to copy. Unmount stops it.	Real Jupyter server as the GPU box
`test_awake`	caffeinate / systemd-inhibit are started with the first mount and stopped after the last; turned off by the setting	Stand-in processes

TEST	WHAT IT PROVES	REAL OR STAND-IN
`test_cloudflare`	Setup makes the tunnel, ingress, DNS record and Argo, and reuses them when run again. Without Argo billing the tunnel still works. The API token is never saved. The tunnel's token goes in the environment, not on the command line.	A stand-in for Cloudflare's API
`scripts/compute-link.test.mjs`	The page's mount calls, their errors, and the words shown	Stand-in fetch
`tests/live_tunnel.py`	The mount over a real tunnel, measured	**Real Cloudflare; not run here** (section 5, step 11)

Totals at the last run: 177 Companion tests pass on Python 3.9 and 3.13 (1 skipped, as before), the page's 75 related tests pass, and `tsc -b` is clean.

8. Which limits are the mount's, and which the tunnel's

A quick test: if a limit would still apply with a cable straight from the laptop to the GPU, it belongs to the mount.

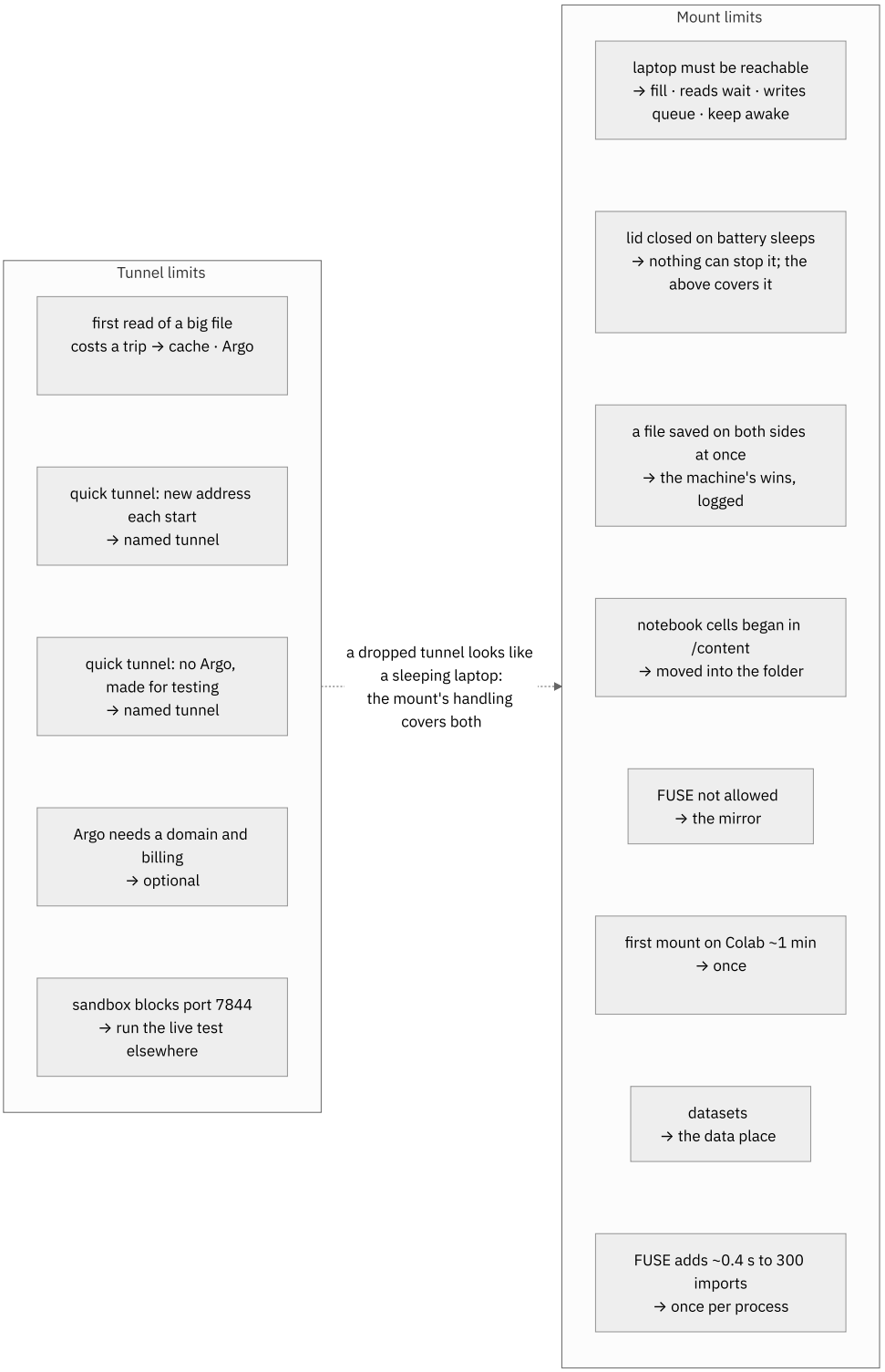


Figure 10. The limits, sorted. Only one can't be engineered away: the code lives on your laptop. For a run of several days with the laptop closed, unmounting (copying) is still the safer choice.

9. Trying it yourself, step by step

9.1 Before you start

1. Merge or check out `claude/epic-darwin-18jc7s` and deploy the site (or run it locally), so the page has the mount switch.

2. Update the Companion on your laptop to **0.19.0**: Settings → Updates, or run the setup line again. `reader-companion --version` should say 0.19.0.

9.2 The tests on your laptop (10 minutes)

```
cd companion
git fetch origin && git checkout claude/epic-darwin-18jc7s   # the mount is on this branch
until #322 is merged
python3 -m venv .venv && . .venv/bin/activate
pip install -e . fusepy
python3 -m unittest discover -s tests          # 177 tests; FUSE ones skip where there's no
/dev/fuse (macOS)
cd .. && npm ci && npx tsc -b && node --test scripts/compute-link.test.mjs
scripts/companion.test.mjs
```

Use `python3` (macOS has no `python` command outside a virtual environment). On macOS, which has no `/dev/fuse` without macFUSE, the FUSE tests skip and the mirror tests still run. On Linux, `sudo apt-get install fuse libfuse2` first.

9.3 A real tunnel, measured (5 minutes)

```
cd companion
python3 -m tests.live_tunnel
```

Expect a `trycloudflare.com` address, "mounted", and then these lines:

- **6 MB cold / cached:** the cold read is your trip to Cloudflare and back, twice.
- **save here → there:** about 0.5 s plus one trip.
- **write there → here:** about one trip.
- **round trip of a read:** in milliseconds.

Paste the JSON back to me and I'll tune the fill and wait settings from it.

9.4 The mount on Colab (15 minutes)

1. Open a project whose files are on your laptop (Storage & compute → **Move files here** if they aren't), with its code on Colab, and connect a GPU runtime.
2. Storage & compute → **Sync** → tick **Mount the folder on Colab**. The first time, wait about a minute while fusepy installs. The bar should then read **Mounted on Colab ⇄ live**.
3. In the compute's terminal: `ls /content/playgrounds/*/` shows your files, and `mount | grep koios` shows the FUSE mount.
4. Edit a `.py` file on your laptop and save it. Within about a second, `cat` it on Colab.
5. In a notebook cell: `import os; os.getcwd()` should be the project's folder, and `import yourmodule` should work.
6. Run something that writes `results/x.json` and `checkpoints/model.pt`. The JSON appears on your laptop; the checkpoint doesn't, and turns up in Drive under `Koios/Data/<project>/` a few seconds later.

7. On macOS, while it is mounted, `pmset -g assertions | grep caffeinate` shows the Companion holding sleep off.
8. Turn Wi-Fi off for a minute and keep running. Reads go on, and a file written meanwhile appears on your laptop once Wi-Fi is back. `cat ~/.koios-mount/*/status.json` on Colab shows `awaySince` and `waitingToWrite`.
9. Untick the switch. The bar goes back to **Send code**, and runs copy again.

9.5 Argo Smart Routing (20 minutes, needs a Cloudflare account)

1. Add a domain to Cloudflare, and set up billing if you want Argo.
2. Make an API token with: Account · Cloudflare Tunnel: Edit; Zone · DNS: Edit; Zone · Zone Settings: Edit; Zone · Zone: Read.
3. `reader-companion tunnel setup --hostname koios.yourdomain.com --api-token <token>`, then `reader-companion start`.
4. `reader-companion tunnel status` should say "named tunnel … Argo Smart Routing on".
5. Measure again with your tunnel, and compare with 9.3:

```
   KOIOS_LIVE_HOSTNAME=koios.yourdomain.com KOIOS_LIVE_TUNNEL_TOKEN=<from ~/.reader-
companion/config.json> python3 -m tests.live_tunnel
```

1. If you decide Argo isn't worth what it costs, turn it off under the domain → Traffic → Argo Smart Routing, or run `reader-companion tunnel quick` to go back to quick tunnels.

9.6 What isn't verified yet

	WHY	HOW IT GETS VERIFIED
The mount over a real Cloudflare tunnel	The sandbox blocks port 7844	9.3
fusepy and libfuse2 installed by the mount on a real Colab runtime	No Colab here	9.4, steps 2 and 3
Argo on a real account	Needs your account and billing	9.5
Keep-awake on a real Mac or Windows PC	Tested with stand-in processes only	9.4, step 7